

Metro Ticket Generation System

Phase 5 — Deployment, Documentation & Final Presentation | ServiceNow

Phase Objective	Prepare the completed solution for deployment, document how it works and can be recreated, and present a clear end-to-end demonstration.
Key Areas	Troubleshooting • Timelines • Innovation • Functional Overview • Technical Blueprint • Setup Manual • Demo Planning • Explanation • Realism • Scalability
Delivery Principle	A solution is complete when it is functional, understandable, reproducible, demonstrable and maintainable.

1. Phase 5 Overview

Phase 5 is the final delivery stage. It resolves remaining issues, confirms adherence to the project roadmap, prepares functional and technical documentation, plans the demo and defines future scalability.

Development → Testing & Security → Troubleshooting → Documentation → Demo Rehearsal → Final Presentation → Future Roadmap

2. Brainstorming & Final Readiness

- What must be checked before deployment?
- Can another developer recreate the solution in a new PDI?
- Can the presenter explain the business purpose before technical details?
- Which screenshots and configuration evidence best demonstrate the solution?
- How can route/fare logic scale beyond the initial sample routes?
- What troubleshooting path should an administrator follow for QR, fare, flow or variable issues?

3. Phase 5 Deliverables

Deliverable	Expected Content
Deployment readiness	Configuration review, dependencies, access validation and final smoke test.
Functional overview	Passenger flow from input to fare, request and QR experience.
Technical blueprint	Tables, variables, scripts, UI policies, Flow Designer and ACL relationships.
Setup manual	Ordered steps to recreate the solution in a fresh PDI.
Demo plan	Narrated end-to-end walkthrough with requester and fulfiller perspectives.
Final presentation	Architecture, implementation, evidence, quality and future roadmap.

4. Troubleshooting

Troubleshooting was handled as structured diagnosis. The documented implementation identifies QR encoding issues, fare mismatches, variable population problems and form-behavior issues as key areas.

Problem Area	Diagnostic / Resolution Approach
QR encoding	Use logs and print statements to inspect QR URL construction and encoding.
Fare mismatch	Trace source, destination, journey type and passenger count; verify configured fare logic.
Flow execution	Use Flow Designer Execution Details to trace steps, failures and condition paths.
Variable population	Check catalog client scripts and variable bindings for missing or misaligned values.
UI behavior	Test UI policies and visibility logic in Service Portal and preview mode.

5. Troubleshooting Decision Flow

Issue → Reproduce → Inspect inputs → Review client/UI logic → Check Flow execution → Validate backend → Fix → Retest → Document

6. Adherence to Timelines

The project followed a milestone-based roadmap: Catalog Creation → Form Setup → Testing & Security → Deployment. Each milestone depends on the previous one, reducing rework and keeping implementation aligned with the planned delivery.

- Catalog creation established the ServiceNow entry point.
- Form setup added variables and user interaction logic.
- Testing & Security validated access, behavior and data integrity.
- Deployment follows completion of validation, documentation and demo readiness.

7. Innovation & Maintainability

- Implemented end-to-end automation using Flow Designer.
- Used dynamic task creation where workflow processing requires it.
- Kept client scripts and UI policies limited to essential behavior.
- Designed the solution for maintainability and easier handoff.
- Combined dynamic fare population with QR-based ticket presentation.

8. Functional Overview

The functional document explains what the system does. The core journey is: select stations → journey type → passenger count → fare calculation → payment mode → checkout → QR experience → request/data processing.

Step	User Action	System Response
1	Open Book A Metro Ticket	Loads catalog form.
2	Select stations	Captures source/destination references.
3	Select journey type	Determines single or return journey.
4	Select passenger count	Triggers dynamic fare calculation.
5	Select payment mode	Others can reveal Enter Payment Mode.
6	Checkout	Triggers configured QR-ticket experience.
7	Continue order	Request proceeds through ServiceNow process.

9. Technical Blueprint

- Catalog layer: Book A Metro Ticket and its variables.
- Data layer: Metro Station Details custom table and related backend records.
- Client logic: Fare calculation and QR-ticket behavior.
- UI logic: Conditional field visibility using Catalog UI Policy.
- Automation: Flow Designer / Process Automation and field mapping.
- Security: ACLs and role-based access.
- Portal: Service Portal provides the user-facing booking experience.

10. Variable-to-Field Mapping

Catalog Variable	Business Data
Starting From	Origin station reference
Going To	Destination station reference
Type of Journey	Single / Return
No of Passengers	Passenger quantity
Amount for Single Journey	Calculated fare
Amount Including Return	Calculated return fare
Mode of Payment	Payment selection
Enter Payment Mode	Additional value when Others is selected

11. Project Architecture Summary

User → Service Catalog → Variables → Client/UI Logic → Validation → Automation/Mapping → Backend Table → Ticket/QR Experience

12. Setup Manual

The setup manual is intended to let another developer recreate the solution in a new Personal Developer Instance (PDI). The following order minimizes dependency issues.

Order	Setup Activity
1	Create Metro Station Details custom table and station data.
2	Create Book A Metro Ticket catalog item and assign catalog/category.
3	Create and configure catalog variables.
4	Configure fare-related Catalog Client Script.
5	Configure Catalog UI Policy and action for Others.
6	Configure QR Client Script and metro_qr_widget integration.
7	Configure Flow Designer / Process Automation and field mapping.
8	Review ACLs and roles.
9	Load/test sample data.
10	Run functional, security and end-to-end smoke tests.

13. Reproducibility Checklist

- Record table/field names exactly.
- Record catalog variable names and internal names.
- Document choice values and reference tables.
- Preserve client-script trigger types and logic.
- Document UI-policy conditions/actions.
- Document Flow Designer inputs, actions and mappings.
- Document ACL expectations and roles.
- Include screenshots at major checkpoints.
- Validate the build in a fresh PDI before handoff.

14. Project Demo Video Planning

The demo should tell a business story rather than only showing configuration screens. Start from the portal and follow a realistic requester journey through ticket creation and QR presentation.

Demo Segment	Narration / Evidence
Introduction	Problem statement and solution objective.
Portal walkthrough	Home → Book A Metro Services → Submit Request.
Live request	Enter stations, journey type and passenger count.
Dynamic behavior	Show fare calculation and conditional payment field.
Automation	Explain processing and data storage at a high level.
QR experience	Show QR ticket and explain its purpose.
Security	Briefly explain ACL-based backend protection.
Conclusion	Summarize outcome and future scope.

15. Explanation Clarity

Documentation and presentation follow a logical What → Why → How → Outcome structure. Each step should include a clear heading, labelled evidence and an explanation that tells the reviewer what changed and why it matters.

- Explain business purpose before configuration detail.
- Explain dependencies before demonstrating dependent features.
- Use consistent names for catalog variables, tables and automation components.
- Show actual outcomes after configuration, not only setup screens.

16. Realism & Quality

The solution reflects a realistic city-metro workflow with station references, passenger-based fare calculation, journey types, payment selection, backend processing and QR-ticket presentation. Realistic sample routes and data make the demonstration meaningful.

- Use realistic station names and route combinations.
- Validate amounts against the configured fare matrix.
- Test normal and exception paths.
- Ensure the demo reflects the documented implementation.

17. Visual Evidence — Supplied Project Screenshots

17. Visual Evidence — Supplied Project Screenshots

The supplied screenshots are included as implementation evidence for the final documentation and presentation.

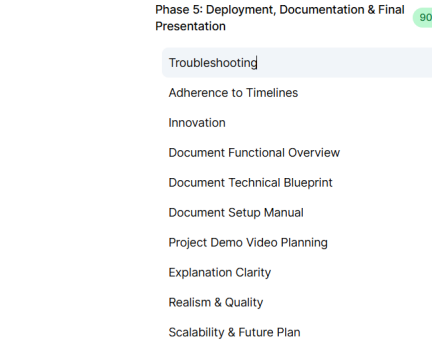


Figure 1. Supplied implementation evidence.

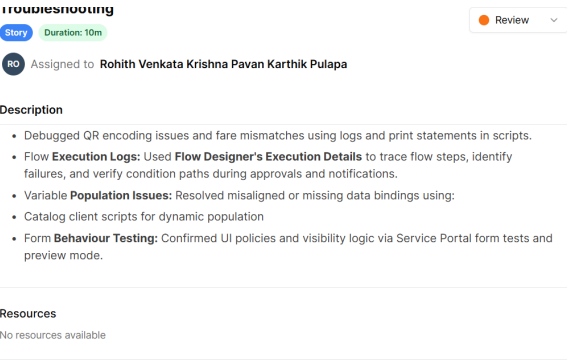


Figure 2. Supplied implementation evidence.

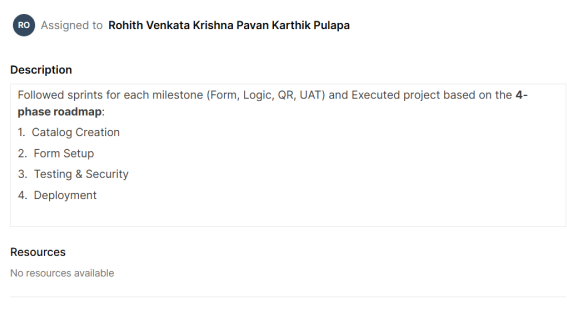


Figure 3. Supplied implementation evidence.

18. Explanation Clarity

Documentation and presentation follow a logical What → Why → How → Outcome structure. Each major step should have a clear heading, labelled evidence and a short explanation of why the configuration matters.

- Explain business purpose before technical configuration.
- Explain dependencies before demonstrating dependent features.
- Use consistent names for variables, tables and automation components.
- Show actual outcomes after configuration, not only setup screens.

19. Additional Visual Evidence

The next set of supplied screenshots provides supporting evidence for configuration, workflow and project presentation.

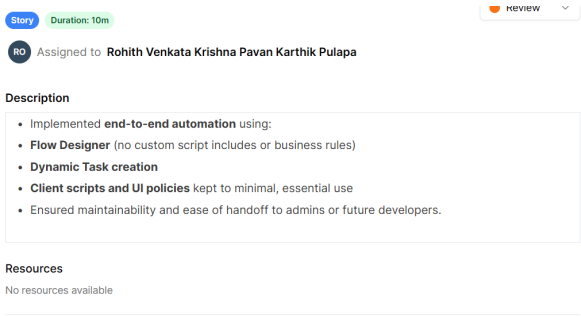


Figure 4. Supplied project evidence.

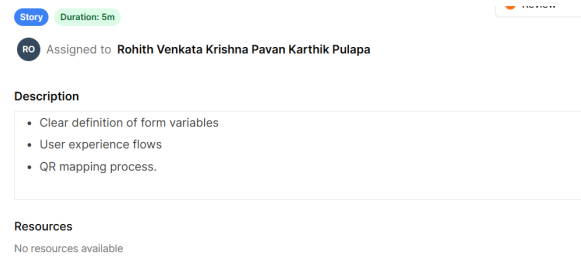


Figure 5. Supplied project evidence.

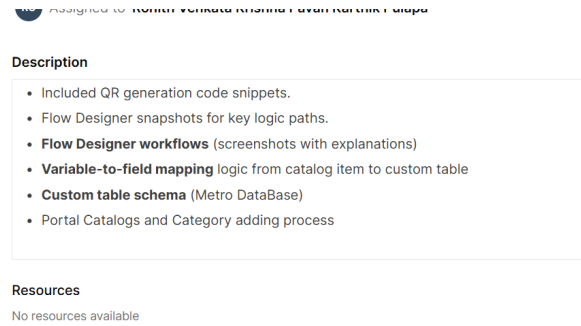


Figure 6. Supplied project evidence.

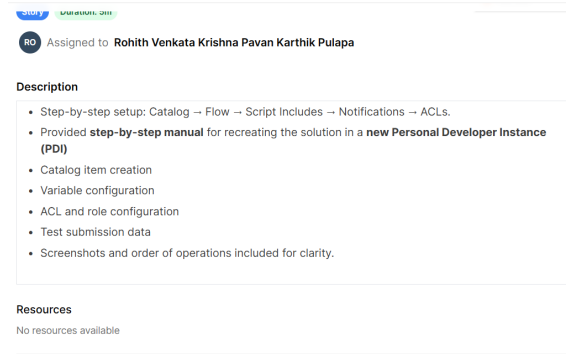


Figure 7. Supplied project evidence.

20. Realism & Quality

The solution reflects a realistic city-metro workflow with station references, passenger-based fare calculation, journey types, payment selection, backend processing and QR-ticket presentation. Realistic sample routes and data make the demonstration meaningful.

- Use realistic station names and route combinations.
- Validate amounts against the configured fare matrix.
- Test normal and exception paths.
- Ensure the demo reflects the documented implementation.

21. Final Evidence & Presentation Checklist

The remaining supplied screenshots are retained as final supporting evidence. They can be referenced during the final presentation to prove implementation and user-flow behavior.

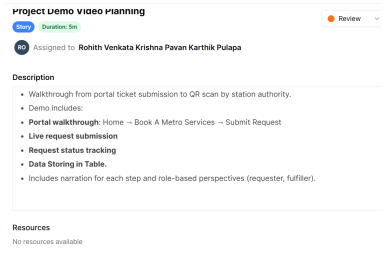


Figure 8. Supplied project evidence.

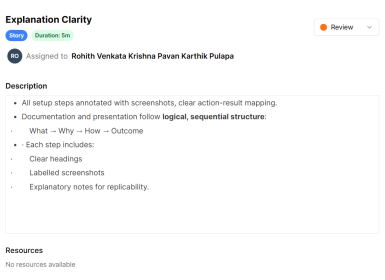


Figure 9. Supplied project evidence.

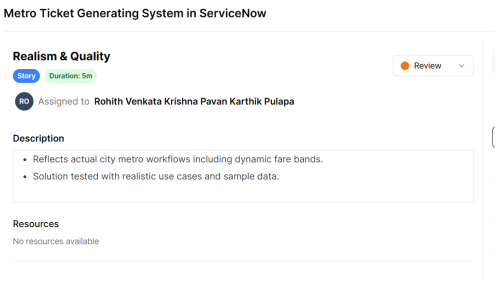


Figure 10. Supplied project evidence.

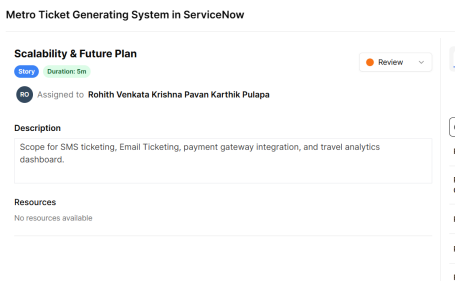


Figure 11. Supplied project evidence.

22. Presentation Quality Checklist

- Start with the business objective and user problem.
- Explain the architecture before deep technical details.
- Demonstrate one complete successful booking.
- Show dynamic fare calculation and conditional payment behavior.
- Show the QR-ticket experience.
- Explain troubleshooting and security decisions.
- Finish with measurable outcome and future roadmap.

23. Scalability & Future Plan

- Extend SMS ticketing for confirmations and travel alerts.
- Add Email Ticketing for digital receipts and ticket delivery.
- Integrate a payment gateway.
- Introduce a travel analytics dashboard for route usage, passenger volume and revenue.
- Move route/fare logic toward maintainable data-driven fare tables and server-side calculation.

- Expand station coverage and configurable fare bands.
- Add stronger ticket verification and ticket lifecycle management.

24. Final Delivery Outcome

Phase 5 completes the delivery lifecycle by turning the Metro Ticket Generation System into a documented, demonstrable and maintainable ServiceNow solution. The project has a structured troubleshooting approach, reproducible setup path, clear demo story and practical future roadmap.

Final Delivery: Functional + Secure + Documented + Reproducible + Demonstrable + Scalable